

LAPORAN AKHIR TUGAS BESAR

KEAMANAN SERVER DAN JARINGAN

Dosen mata kuliah: Ainul Hizriadi S.Kom., M.Sc



Disusun Oleh:

Kelompok 1

Ahmad Arif Fatahillah	241402036
Rahma Sarita Nasution	241402043
Agnes Olivia Ketaren	241402066
Richard Lim	241402073
Davina Caecilia Marpaung	241402076
Bryan Sullivan Nauli	241402108
Jelita Amy Syafira Rangkuti	241402117

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS ILMU KOMPUTER DAN
TEKNOLOGI INFORMASI
UNIVERSITAS SUMATERA UTARA

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan akhir tugas besar yang berjudul Keamanan Server dan Jaringan ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas mata kuliah Keamanan Server dan Jaringan pada Program Studi Teknologi Informasi, Fakultas Ilmu Komputer dan Teknologi Informasi, Universitas Sumatera Utara.

Dalam penyusunan laporan ini, kami melakukan perancangan, konfigurasi, dan implementasi sistem layanan web yang aman menggunakan lingkungan Linux dan teknologi containerization. Berbagai mekanisme keamanan diterapkan, di antaranya konfigurasi SSH Hardening, pengamanan jaringan menggunakan UFW dan Fail2Ban, penerapan SSL/TLS untuk komunikasi terenkripsi, serta deployment layanan berbasis Docker dengan arsitektur tiga kontainer yang terdiri atas Nginx sebagai reverse proxy, WordPress sebagai application server, dan MariaDB sebagai database server. Selain implementasi sistem, laporan ini juga membahas desain arsitektur, proses konfigurasi, pengujian keamanan, serta verifikasi terhadap setiap mekanisme keamanan yang diterapkan.

Kami menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi pembahasan. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa yang akan datang. Akhir kata, kami berharap laporan ini dapat memberikan manfaat dan menambah wawasan mengenai penerapan keamanan server dan jaringan, khususnya dalam implementasi layanan web yang aman, andal, dan terisolasi menggunakan teknologi Linux dan Docker.

Medan, 29 Mei 2026

Kelompok 1

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang	1
1.2. Tujuan	1
1.3. Ruang Lingkup.....	1
BAB II DESAIN ARSITEKTUR SISTEM.....	3
2.1. Desain Sistem.....	3
2.2. Diagram Jaringan	4
BAB III KONFIGURASI DAN IMPLEMENTASI	5
3.1. Konfigurasi Penggantian Port SSH (SSH Hardening).....	5
3.2. Persiapan Lingkungan Kerja dan Keamanan Dasar.....	7
3.3. Pengaturan Kredensial Keamanan	10
3.4. Konfigurasi SSL/TLS dan Reverse Proxy	11
3.5. Deployment dengan Docker Compose	12
BAB IV HASIL UJI KEAMANAN DAN VERIFIKASI.....	14
4.1. Tes Pengalihan (Redirect) HTTP ke HTTPS	14
4.2. Tes Serangan Brute-Force dan Verifikasi Fail2Ban	14
4.3. Tes Firewall dan Isolasi Database (Port 3306)	15
4.4. Tes URL Manipulation	15
BAB V KENDALA DAN SOLUSI	17
BAB VI PENUTUP	19
6.1. Kesimpulan	19
LAMPIRAN.....	20

BAB I

PENDAHULUAN

1.1.Latar Belakang

Dalam era digital saat ini, keamanan server dan jaringan merupakan aspek fundamental dalam menyediakan layanan web yang andal. Ancaman siber yang terus berkembang menuntut adanya perlindungan berlapis yang diterapkan secara komprehensif, mulai dari tingkat sistem operasi, jaringan, hingga aplikasi. Sebuah peladen (*server*) yang dibiarkan dengan konfigurasi bawaan sangat rentan terhadap serangan otomatis seperti *brute-force* maupun eksploitasi celah keamanan aplikasi.

Oleh karena itu, melalui proyek mata kuliah Keamanan Server & Jaringan ini, pendekatan keamanan tidak hanya dilakukan secara konvensional, tetapi juga mengintegrasikan teknologi *containerization* (kontainerisasi). Penggunaan kontainer memungkinkan adanya isolasi antar-komponen layanan secara spesifik. Dengan memisahkan setiap lingkungan layanan ke dalam jaringan yang terisolasi, risiko pergerakan lateral dari peretas (apabila salah satu layanan berhasil ditembus) dapat diminimalisasi. Proyek ini mendemonstrasikan implementasi praktik terbaik (*best practices*) dalam merancang infrastruktur web yang tidak hanya fungsional, tetapi juga aman dari berbagai potensi eksploitasi.

1.2.Tujuan

Berdasarkan panduan tugas proyek, tujuan dari pelaksanaan konfigurasi ini adalah sebagai berikut.

1. Membangun layanan web yang aman serta tercontainerisasi pada lingkungan *Virtual Machine* Linux.
2. Menguasai perintah-perintah konfigurasi Linux, termasuk manajemen pengguna, hak akses (*file permission*), dan manajemen layanan.
3. Menginstal dan mengkonfigurasi layanan server beserta sistem pengamanan di Linux, seperti pengaturan *firewall*, penerapan SSL/TLS, dan penguatan sistem operasi (*OS hardening*).
4. Menerapkan implementasi *image* dan *containerization* dengan pemisahan kontainer untuk aplikasi, *database*, dan layanan reverse proxy secara tepat.

1.3.Ruang Lingkup

Batasan masalah dan ruang lingkup dari implementasi proyek "Topik A" ini mencakup beberapa hal, antara lain sebagai berikut.

1. Arsitektur Sistem

Mendeploy aplikasi web menggunakan arsitektur 3-kontainer yang terdiri dari Nginx (sebagai *reverse proxy*), *App Server* (WordPress), dan *Database* (MariaDB).

2. **Keamanan Sistem Operasi (OS Hardening)**
Melakukan konfigurasi SSH tingkat lanjut yang mewajibkan penggunaan *Key-only auth* (*SSH key pair*), mengganti *port* standar, menonaktifkan fitur autentikasi *password*, dan menonaktifkan akses *login root*.
3. **Keamanan Jaringan**
Menerapkan UFW (*Uncomplicated Firewall*) dengan kebijakan bawaan (*policy default*) menolak lalu lintas masuk (*deny incoming*), serta mengaktifkan Fail2Ban untuk mencegah serangan *brute-force*.
4. **Keamanan Kontainer**
Menerapkan isolasi menggunakan *Docker bridge network* yang terpisah per *tier*, memastikan kontainer berjalan sebagai *non-root user*, dan memastikan bahwa *port database* tidak dapat diekspos ke *host* luar. Kredensial sistem juga wajib disimpan secara aman menggunakan *file .env*.
5. **Enkripsi Data**
Mewajibkan penggunaan HTTPS (port 443) dengan sertifikat SSL/TLS yang di-*mount* ke kontainer Nginx, disertai mekanisme *redirect* otomatis dari lalu lintas HTTP (port 80).

BAB II

DESAIN ARSITEKTUR SISTEM

2.1. Desain Sistem

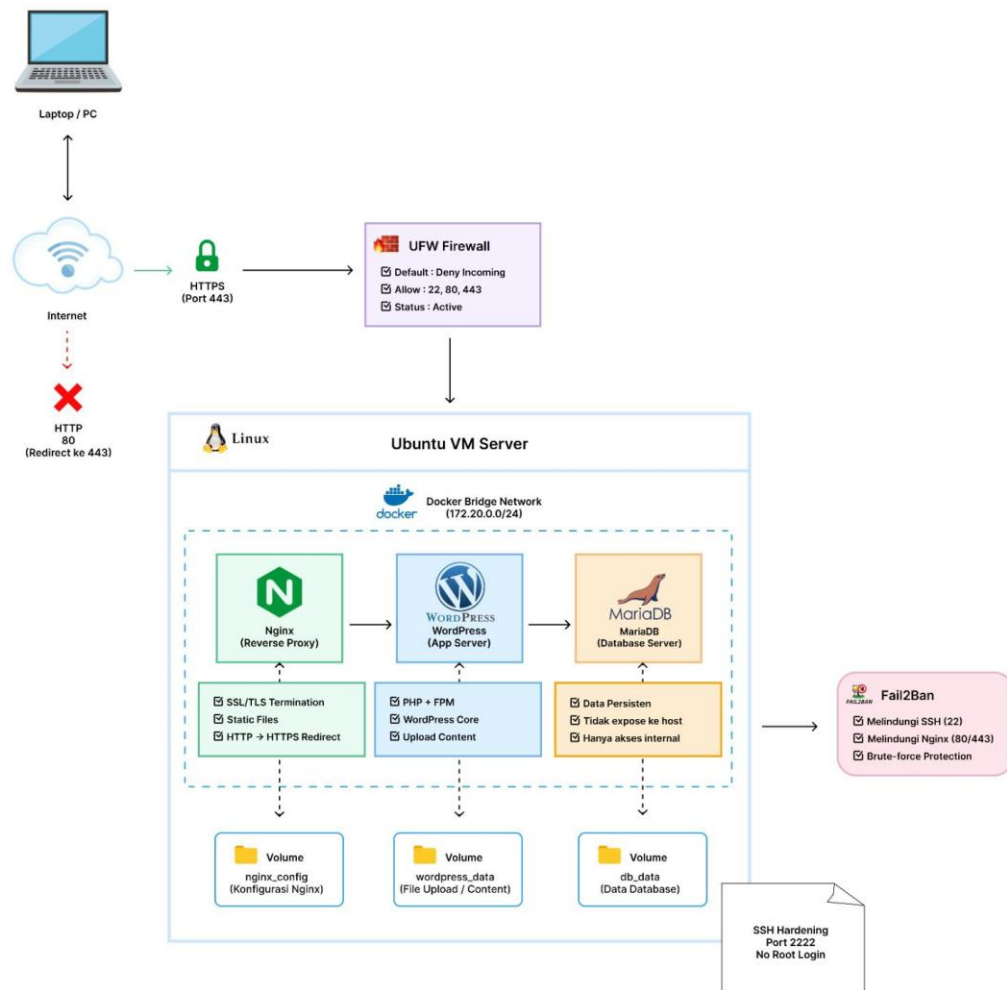
Sistem layanan web pada proyek ini dibangun menggunakan arsitektur berbasis kontainer (*containerized architecture*) yang memisahkan setiap komponen operasional ke dalam lingkungannya masing-masing. Sesuai dengan spesifikasi keamanan untuk Topik A, implementasi ini menggunakan arsitektur *3-tier* yang terdiri dari peladen proksi balik (*reverse proxy*), peladen aplikasi (*app server*), dan peladen basis data (*database server*).

Komponen utama penyusun arsitektur sistem ini dirancang dengan pembatasan hak akses dan isolasi jaringan sebagai berikut:

1. Nginx (Reverse Proxy & SSL Termination)
Bertindak sebagai gerbang terdepan yang menerima seluruh permintaan dari pengguna internet. Kontainer ini dikonfigurasi untuk mengelola sertifikat SSL/TLS yang disematkan (*mounted*) ke dalam kontainer melalui fitur *volume*. Nginx secara otomatis akan menolak atau mengalihkan (*redirect*) seluruh lalu lintas HTTP standar di *port* 80 menuju jalur enkripsi HTTPS di *port* 443, sebelum meneruskannya ke peladen aplikasi.
2. WordPress (App Server)
Berperan sebagai peladen aplikasi utama yang memproses logika sistem dan memuat antarmuka web. Demi keamanan, kontainer ini tidak terekspos secara langsung ke internet, melainkan hanya menerima lalu lintas internal yang diteruskan dari kontainer Nginx melalui jaringan isolasi Docker (*bridge network*).
3. MariaDB (Database Server)
Berfungsi sebagai pusat penyimpanan data aplikasi. Kontainer basis data ini diletakkan pada tingkat isolasi jaringan terdalam dan terpisah. *Port* basis data (3306) sama sekali tidak diekspos (*publish*) ke sistem operasi *host* (Ubuntu), sehingga mustahil diakses dari luar *server* dan hanya bisa diakses secara internal oleh peladen WordPress.

Selain pemisahan pada tingkat kontainer, sistem secara keseluruhan dilindungi oleh lapisan keamanan pada tingkat sistem operasi *host* menggunakan UFW (*Uncomplicated Firewall*) dengan aturan *default deny incoming*, serta Fail2Ban untuk memblokir alamat IP yang terdeteksi melakukan serangan *brute-force*. Pengaturan kredensial yang sensitif juga dipisahkan pengelolaannya melalui file *.env* untuk menghindari praktik *hardcoding*.

2.2. Diagram Jaringan



BAB III

KONFIGURASI DAN IMPLEMENTASI

3.1. Konfigurasi Penggantian Port SSH (SSH Hardening)

Konfigurasi SSH dilakukan dengan mengubah port default SSH (22) menjadi port kustom (2222) untuk mengurangi risiko serangan otomatis (*brute-force*). Pada sistem berbasis Ubuntu versi terbaru, layanan SSH menggunakan mekanisme *socket activation* dari systemd, sehingga pengubahan port dilakukan pada konfigurasi *socket*, bukan hanya pada file `sshd_config`

```
arif@pksj:~$ sudo ss -tulpn | grep 2222
arif@pksj:~$ sudo mkdir -p /etc/systemd/system/ssh.socket.d
```

Di awal, dilakukan pengecekan status dan port yang menunjukkan port 2222 belum aktif. Selanjutnya, dibuat direktori baru yang berfungsi untuk menyimpan file konfigurasi kustom (*drop-in file*) yang akan menimpa konfigurasi bawaan `ssh.socket` tanpa mengubah file aslinya.

```
arif@pksj:~$ echo -e "[Socket]\nListenStream=\nListenStream=0.0.0.0:2222" | sudo
tee /etc/systemd/system/ssh.socket.d/override.conf
[Socket]
ListenStream=
ListenStream=0.0.0.0:2222
```

Perintah `echo` dijalankan untuk membuat file `override.conf` dan langsung mengisinya dengan konfigurasi port yang baru. Di dalam file tersebut, baris `'ListenStream='` bertugas untuk menghapus aturan *listening port* default (port 22) dan baris `'ListenStream=0.0.0.0:2222'` berfungsi agar layanan SSH mendengarkan lalu lintas masuk melalui port 2222 pada semua antarmuka jaringan IPv4. Perintah `echo` dijalankan untuk membuat file `override.conf` dan langsung mengisinya dengan konfigurasi port yang baru. Di dalam file tersebut, baris `'ListenStream='` bertugas untuk menghapus aturan *listening port* default (port 22) dan baris `'ListenStream=0.0.0.0:2222'` berfungsi agar layanan SSH mendengarkan lalu lintas masuk melalui port 2222 pada semua antarmuka jaringan IPv4.

```
arif@pksj:~$ sudo systemctl daemon-reload
arif@pksj:~$ sudo systemctl restart ssh.socket
```

Layanan kemudian akan di-*restart* agar konfigurasi dimuat ulang dan *socket* SSH dapat menggunakan layanan port yang baru. Langkah terakhir yang harus dilakukan adalah memverifikasi apakah port telah berhasil diganti dengan menjalankan perintah `sudo ss -tulpn | grep 2222`.

```
arif@pksj:~$ sudo ss -tulpn | grep 2222
tcp    LISTEN 0      4096      0.0.0.0:2222      0.0.0.0:*      users:((("sshd",
pid=4653,fd=3),("systemd",pid=1,fd=87))
```

Output pada terminal (tcp LISTEN... 0.0.0.0:2222) membuktikan bahwa systemd kini telah berhasil *listening* secara aktif pada port 2222 untuk layanan sshd.

```
C:\Users\Arif>ssh -p 2222 ubuntu@172.30.67.104
ubuntu@172.30.67.104's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-110-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May 25 04:47:07 AM UTC 2026

System load:  0.0          Processes:            127
Usage of /:   14.3% of 47.93GB Users logged in:          1
Memory usage: 10%         IPv4 address for ens18: 172.30.67.104
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

43 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

*** System restart required ***
Last login: Fri May 22 07:57:16 2026 from 10.205.8.17
ubuntu@ubuntu:~$ |
```

Selanjutnya dilakukan pembuatan pasangan kunci SSH (*SSH Keypair*) pada mesin lokal. Pertama-tama akan dilakukan pembuatan kunci SSH dengan menjalankan perintah `ssh-keygen -t rsa -b 4096`.

```
ssh-keygen -t rsa -b 4096
```

Selama eksekusi perintah tersebut, sistem akan memunculkan tiga prompt pertanyaan yang meminta pengguna menentukan lokasi penyimpanan berkas dan mengatur *passphrase* (kata sandi tambahan). Tindakan ini berfungsi untuk menetapkan penyimpanan berkas kunci pada direktori bawaan sistem operasi serta meniadakan penggunaan *passphrase*, sehingga memfasilitasi proses integrasi berkelanjutan, seperti *login* ke peladen atau sinkronisasi Git, agar dapat berjalan otomatis di latar belakang tanpa interupsi permintaan kata sandi.

```
type %USERPROFILE%\ssh\id_rsa.pub
```

Selanjutnya, perintah diatas dijalankan untuk mengekstraksi dan menampilkan isi berkas kunci publik (`id_rsa.pub`) ke layar terminal. Untaian sandi ini kemudian disalin untuk dimasukkan ke dalam berkas `authorized_keys` di server tujuan, agar mesin lokal diotorisasi untuk melakukan akses secara aman.

Untuk menyempurnakan keamanan, autentikasi menggunakan kata sandi

(password) dinonaktifkan sepenuhnya. Hal ini dilakukan dengan mengubah nilai `PasswordAuthentication yes` menjadi `no` pada berkas konfigurasi `/etc/ssh/sshd_config` di server, lalu melakukan *restart* pada layanan SSH. Dengan demikian, server hanya akan menerima koneksi dari mesin lokal yang memiliki kunci privat yang sah.

```
Port 2222
```

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

```
/etc/ssh/sshd_config
```

3.2. Persiapan Lingkungan Kerja dan Keamanan Dasar

Tahap awal *deployment* sistem dimulai dengan mempersiapkan repositori dan ruang kerja pada *Virtual Machine* (VM). Repositori proyek terlebih dahulu dikloning ke dalam sistem lokal menggunakan perintah `git clone`. Setelah itu, struktur direktori diorganisasikan dengan membuat *folder* docs, docker, dan scripts secara terpisah guna mengelompokkan berkas dokumentasi, konfigurasi kontainer, dan skrip instalasi.

```
ubuntu@ubuntu:~$ git clone https://github.com/richardlcp119/kel-01-topik-a.git
Cloning into 'kel-01-topik-a'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
ubuntu@ubuntu:~$ sudo systemctl restart ssh
```

```
ubuntu@ubuntu:~/kel-01-topik-a$ mkdir scripts
ubuntu@ubuntu:~/kel-01-topik-a$ nano scripts/setup.sh
ubuntu@ubuntu:~/kel-01-topik-a$ chmod +x scripts/setup.sh
ubuntu@ubuntu:~/kel-01-topik-a$ ./scripts/setup.sh
```

Sebagai upaya efisiensi dan konsistensi dalam tahap pra-konfigurasi server, proses instalasi dependensi perangkat lunak dan penerapan lapisan keamanan dasar diotomatisasi menggunakan skrip *bash*. File `setup.sh` dibuat di dalam direktori `scripts` dan diberikan hak akses eksekusi melalui perintah `chmod +x`.

```
echo "Memperbarui sistem..."
sudo apt update && sudo apt upgrade -y

echo "Menginstal Docker & Docker Compose..."
sudo apt install docker.io docker-compose -y
sudo systemctl enable docker
sudo systemctl start docker

echo "Mengonfigurasi Firewall (UFW)..."
sudo apt install ufw -y
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 2222/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable

echo "Menginstal & Mengaktifkan Fail2Ban..."
sudo apt install fail2ban -y
sudo systemctl enable fail2ban
sudo systemctl start fail2ban

echo "Setup Infrastruktur Selesai & Aman!"

/scripts/setup.sh
```

Saat skrip dieksekusi, sistem secara otomatis menjalankan serangkaian tahapan infrastruktur secara berurutan, yang meliputi:

1. OS Update & Upgrade
Skrip secara otomatis memperbarui daftar repositori dan mengupdate seluruh paket bawaan Ubuntu (apt update && apt upgrade -y). Langkah ini merupakan praktik fundamental untuk memastikan sistem operasi terbebas dari kerentanan (*vulnerability*) pada perangkat lunak versi lama sebelum layanan berjalan.
2. Instalasi Mesin Kontainer (*Container Engine*)
Meningat arsitektur layanan dibangun dalam lingkungan yang terisolasi, skrip menginstal docker.io beserta pelengkap orkestrasinya, docker-compose. Layanan *daemon* Docker kemudian diatur agar otomatis menyala setiap kali sistem melakukan *booting* (systemctl enable docker).
3. Konfigurasi Awal Firewall (UFW)
Sistem keamanan jaringan diatur dengan memasang *Uncomplicated Firewall* (UFW). Skrip mengonfigurasi regulasi dasar menjadi *default deny incoming* untuk memblokir seluruh koneksi asing dan *default allow outgoing* agar sistem server memiliki izin bebas untuk menginisiasi koneksi keluar, seperti kebutuhan mengambil pembaruan repositori OS atau mengunduh *image* kontainer dari

luar. Jalur komunikasi dari luar kemudian dibuka secara selektif hanya untuk port 2222 (akses administrasi *remote* SSH), port 80 (HTTP), dan port 443 (HTTPS).

4. Pemasangan Sistem Pencegahan Intrusi

Skrip menginstal Fail2Ban dan langsung mengaktifkannya sebagai pelindung tambahan. Layanan ini akan memantau log aktivitas sistem dan secara otomatis memblokir alamat IP yang terdeteksi melakukan serangan *brute-force* atau percobaan otentikasi yang gagal secara berulang.

```
ubuntu@ubuntu:~/kel-01-topik-a$ ./scripts/setup.sh
Updating system...
[sudo] password for ubuntu:
Hit:1 http://id.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://id.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://id.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://id.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:6 http://id.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:7 http://id.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:8 http://id.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5,744 B]
Get:9 http://id.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [42.5 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.3 kB]
Fetched 1,076 kB in 1s (847 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
43 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  apparmor cloud-init coreutils distro-info-data fwupd iproute2 libapparmor1 libfwupd2 libheif-plugin-aomdec libheif-plugin-aomenc
  libheif-plugin-libde265 libheif1 libnetplan1 libnftables1 libnss-systemd libpam-systemd libsystemd-shared libsystemd0 libudev1
  linux-base linux-firmware lshw netplan-generator netplan.io nftables open-vm-tools python3-netplan python3-software-properties
  rsyslog snapd software-properties-common sosreport systemd systemd-dev systemd-hwe-hwdb systemd-resolved systemd-sysv
  systemd-timesyncd thermald ubuntu-drivers-common ubuntu-pro-client ubuntu-pro-client-l10n udev
43 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 697 MB of archives.
```

```
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.8.0-110-generic
Diagnostics:
  The currently running kernel version is not the expected kernel version 6.8.0-117-generic.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #1: login[841]
ubuntu @ session #5300: sshd[102773]
ubuntu @ session #5305: sshd[103097]
ubuntu @ session #5316: sshd[103594]
ubuntu @ session #5317: sshd[103726]
ubuntu @ session #5319: sshd[103801]
ubuntu @ session #5320: sshd[103910]
ubuntu @ user manager service: systemd[964]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Setup Selesai!
```

Setelah *script* selesai berjalan, langkah selanjutnya adalah memverifikasi instalasi perangkat lunak utama yang dibutuhkan.

```
ubuntu@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
2222/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
2222/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)

ubuntu@ubuntu:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-09 06:46:29 UTC; 2h 45min ago
     Docs: man:fail2ban(1)
    Main PID: 189466 (fail2ban-server)
      Tasks: 5 (limit: 4600)
    Memory: 20.1M (peak: 20.8M)
       CPU: 11.009s
    CGroup: /system.slice/fail2ban.service
            └─189466 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Jun 09 06:46:29 ubuntu systemd[1]: Started fail2ban.service - Fail2Ban Service.
Jun 09 06:46:29 ubuntu fail2ban-server[189466]: 2026-06-09 06:46:29,682 fail2ban.configreader [189466]: WARNING 'allowipv6' not defined in 'Defin
Jun 09 06:46:29 ubuntu fail2ban-server[189466]: Server ready
```

3.3. Pengaturan Kredensial Keamanan

Sebagai bagian dari implementasi keamanan aplikasi dan perlindungan data, pengelolaan kredensial sensitif seperti kata sandi tidak dilakukan secara langsung di dalam berkas konfigurasi utama. Kredensial sistem disimpan secara terpisah menggunakan berkas lingkungan atau `.env`.

Proses konfigurasi kredensial diawali dengan menyalin template berkas lingkungan yang telah disiapkan di dalam repositori, yaitu dengan menggandakan file `.env.example` menjadi `.env`. Berkas `.env.example` ini berfungsi sebagai kerangka referensi yang memuat daftar variabel yang dibutuhkan oleh sistem, namun sengaja dibuat sebagai templat tanpa menyertakan nilai asli yang bersifat sensitif.

```
cp .env.example .env
```

Penggunaan berkas `.env` ini secara spesifik bertujuan untuk mengisolasi kredensial basis data yang digunakan oleh layanan MariaDB dan peladen aplikasi WordPress. Dengan memisahkan pendefinisian variabel lingkungan, informasi rahasia—seperti kata sandi `root` basis data, nama pengguna, dan kata sandi pengguna untuk WordPress—tidak akan tertulis secara eksplisit (*hardcoded*) di dalam fail `docker-compose.yml`.

```

MYSQL_ROOT_PASSWORD=ganti_dengan_password_kuat
MYSQL_DATABASE=wordpress_db
MYSQL_USER=wp_user
MYSQL_PASSWORD=ganti_dengan_password_kuat

WORDPRESS_DB_HOST=mariadb:3306
WORDPRESS_DB_NAME=wordpress_db
WORDPRESS_DB_USER=wp_user
WORDPRESS_DB_PASSWORD=ganti_dengan_password_kuat

DOMAIN=example.com

```

.env

3.4. Konfigurasi SSL/TLS dan Reverse Proxy

Untuk mengamankan jalur komunikasi data, layanan web dikonfigurasi agar beroperasi menggunakan protokol HTTPS. Mengingat sistem dibangun dan berjalan di lingkungan *Virtual Machine* (VM) lokal, sertifikat keamanan dibuat secara mandiri (*Self-Signed Certificate*) dengan memanfaatkan utilitas terintegrasi OpenSSL.

```

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout docker/nginx/ssl/nginx.key \
-out docker/nginx/ssl/nginx.crt \
-subj "/CN=172.30.67.104"

```

Agar perlindungan lalu lintas jaringan berjalan optimal, sistem harus memastikan tidak ada pengguna atau klien yang dapat mengakses situs melalui jalur yang tidak terenkripsi. Oleh karena itu, diterapkan mekanisme *redirect* otomatis untuk memblokir akses HTTP biasa. Modifikasi dilakukan pada berkas konfigurasi *reverse proxy* Nginx (default.conf).

```

server {
    listen 80;
    server_name _;

    return 301 https://$host$request_uri;
}

// ...

```

/docker/nginx/default.config

Melalui direktif `return 301`, Nginx akan mencegat setiap permintaan HTTP biasa dan secara permanen mengalihkannya (*HTTP 301 Moved Permanently*) ke URL yang sama dengan menggunakan skema HTTPS. Konfigurasi ini memaksa seluruh interaksi untuk melewati *port* 443 yang aman.

Selanjutnya pada *server block* utama yang beroperasi di *port* 443, Nginx dikonfigurasi untuk memuat sertifikat SSL/TLS yang telah dibuat sebelumnya dan bertindak sebagai *reverse proxy*. Nginx menangkap seluruh lalu lintas yang telah berhasil didekripsi, lalu meneruskannya (*proxy_pass*) ke kontainer peladen aplikasi di ranah jaringan internal (diidentifikasi sebagai *http://app:80*).

Agar *backend* di Nginx dapat mengenali identitas asli klien, Nginx melakukan injeksi data pada *header* HTTP. Informasi seperti alamat IP asli pengguna dan protokol awal yang digunakan disisipkan ke dalam *header*. Langkah ini memastikan bahwa peladen aplikasi dapat memproses log dan merespons permintaan klien secara akurat seolah-olah berkomunikasi secara langsung, meskipun sebenarnya berada di balik lapisan *proxy* yang terisolasi.

```
// ...

server {
    listen 443;
    server_name _;

    ssl_certificate /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key /etc/nginx/ssl/privkey.pem;

    location / {
        proxy_pass http://app:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

/docker/nginx/default.config

3.5. Deployment dengan Docker Compose

Layanan web dibangun menggunakan arsitektur tiga kontainer yang terpisah, mencakup Nginx sebagai *reverse proxy*, WordPress sebagai *App Server*, dan MariaDB sebagai basis data. Proses penyebaran (*deployment*) dan manajemen orkestrasi kontainer ini dieksekusi menggunakan file *docker-compose.yml*.

```
ubuntu@ubuntu:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
ubuntu@ubuntu:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1
```

Pada tahap awal *deployment*, perintah *sudo docker compose up -d* dijalankan untuk mengunduh (*pull*) image yang dibutuhkan, yaitu *mariadb:latest*, *wordpress:latest*, dan *nginx:latest*, serta membangun ekosistem layanan secara otomatis di latar belakang (*detached mode*).

```

ubuntu@ubuntu:~/kel-01-topik-a/docker$ sudo docker compose up -d
WARN[0000] /home/ubuntu/kel-01-topik-a/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[*] Running 6/6
✓ Container a83151f93574_docker_db_1 Re... 0.2s
✓ Container 4d28d0828056_docker_nginx_1 Recreated 0.2s
✓ Container docker_app_1 Recreated 1.3s
✓ Container docker-db-1 Started 0.2s
✓ Container docker-nginx-1 Started 0.2s
✓ Container docker-app-1 Started 0.2s

```

Bersamaan dengan berjalannya proses *deployment*, Docker secara otomatis membuat sebuah jaringan khusus berjenis *bridge* (secara default bernama sesuai nama folder proyek). Implementasi *bridge network* ini berperan krusial untuk mengisolasi lalu lintas komunikasi antar-kontainer agar terpisah dari jaringan *host* utama, sehingga MariaDB dan WordPress dapat saling berkomunikasi secara internal tanpa harus terekspos ke internet.

Terakhir, pengecekan status operasional dilakukan melalui perintah `docker ps` dan `docker network ls`.

```

ubuntu@ubuntu:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
23f27df2a8d4   wordpress:latest "docker-entrypoint.s..." 10 days ago   Up 10 days   80/tcp
e3fec0d55ecf   docker-app-1   "/docker-entrypoint..." 10 days ago   Up 10 days   0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
c8c69c324411   mariadb:latest "docker-entrypoint.s..." 10 days ago   Up 10 days   3306/tcp
ubuntu@ubuntu:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
2086bfb04f73   bridge   bridge    local
872bd5c6d6d4   docker_default bridge    local
1a84a5d6d5da   docker_wp_network bridge    local
9a608220870b   host     host      local
b22f75197594   none     null      local

```

Hasil output pada terminal memverifikasi bahwa ketiga layanan telah beroperasi (status Up) dan mematuhi regulasi keamanan port. Kontainer Nginx dikonfigurasi sebagai pintu masuk utama dan satu-satunya yang diizinkan untuk memetakan lalu lintas dari port publik (80 dan 443) ke dalam sistem, sementara kontainer App dan DB dikonfigurasi untuk hanya beroperasi secara tertutup di ranah internal *bridge network*.

BAB IV

HASIL UJI KEAMANAN DAN VERIFIKASI

4.1. Tes Pengalihan (Redirect) HTTP ke HTTPS

Pengujian pertama dilakukan untuk memverifikasi mekanisme pengamanan lalu lintas web. Melalui *Command Prompt* di mesin klien, perintah `curl.exe -I http://172.30.67.104` dieksekusi untuk melihat *header* respons dari peladen web saat diakses melalui jalur HTTP standar (tidak terenkripsi).

```
C:\Users\USER>curl.exe -I http://172.30.67.104
HTTP/1.1 301 Moved Permanently
Server: nginx/1.31.1
Date: Tue, 09 Jun 2026 09:54:48 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://172.30.67.104/
```

Output terminal menampilkan respons HTTP/1.1 301 Moved Permanently dengan parameter Location: https://172.30.67.104/. Hasil ini membuktikan bahwa konfigurasi *reverse proxy* Nginx telah berfungsi dengan tepat karena sistem secara otomatis menolak lalu lintas HTTP dan memaksa klien untuk beralih ke jalur komunikasi terenkripsi HTTPS.

4.2. Tes Serangan Brute-Force dan Verifikasi Fail2Ban

Pengujian kedua difokuskan pada perlindungan otentikasi peladen (*OS Hardening*). Skenario ini dilakukan dengan menyimulasikan upaya peretasan atau serangan *brute-force* terhadap layanan SSH.

Pada tahap awal simulasi, mesin klien mencoba masuk secara paksa menggunakan nama pengguna yang tidak valid (penyusup) melalui *port* kustom 2222. Sistem merespons dengan menolak akses (*Permission denied*) secara berulang pada setiap percobaan otentikasi yang gagal.

```
C:\Users\USER>ssh -p 2222 penyusup@172.30.67.104
penyusup@172.30.67.104's password:
Permission denied, please try again.
penyusup@172.30.67.104's password:
Permission denied, please try again.
penyusup@172.30.67.104's password:
penyusup@172.30.67.104: Permission denied (publickey,password).
```

Untuk memastikan bahwa sistem keamanan merespons ancaman tersebut, dilakukan verifikasi log melalui perintah `sudo fail2ban-client status sshd` di sisi peladen. *Output* yang dihasilkan menunjukkan indikator Total failed: 4. Hasil ini mengonfirmasi bahwa layanan Fail2Ban berhasil mendeteksi dan mencatat anomali kegagalan *login*

pada layanan SSH tersebut, membuktikan bahwa sistem pemantauan dan pencegahan intrusi telah beroperasi secara aktif untuk melindungi peladen.

```
ubuntu@ubuntu:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 1
| |- Total failed: 4
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned: 0
  `-- Banned IP list:
```

4.3. Tes Firewall dan Isolasi Database (Port 3306)

Pengujian ini bertujuan untuk memvalidasi efektivitas aturan *Uncomplicated Firewall* (UFW) dan isolasi jaringan kontainer (*Docker bridge network*). Fokus utama pengujian adalah memastikan bahwa layanan basis data (MariaDB) terlindungi secara penuh dan tidak terekspos ke jaringan publik.

```
PS C:\Users\user> Test-NetConnection -ComputerName 172.30.67.104 -Port 3306
WARNING: TCP connect to (172.30.67.104 : 3306) failed
```

Simulasi dilakukan dari mesin klien dengan mencoba membuka koneksi *Transmission Control Protocol* (TCP) menuju alamat IP peladen pada *port* 3306, yang merupakan *port* operasional layanan MariaDB.

4.4. Tes URL Manipulation

Pengujian manipulasi *Uniform Resource Locator* (URL) atau *forced browsing* dieksekusi untuk mengevaluasi kerentanan peladen web terhadap potensi kebocoran informasi (*Information Disclosure*). Skenario ini menyimulasikan taktik peretas yang secara aktif mencoba menebak dan mengakses direktori atau berkas sistem yang seharusnya tidak dapat dijangkau oleh publik.

Pengujian dilakukan dengan memodifikasi URL secara manual pada peramban web klien, yaitu dengan menambahkan *path* direktori tersembunyi seperti */.env* (berkas kredensial variabel lingkungan) dan */.git* (berkas konfigurasi dan riwayat repositori). Sistem merespons upaya tersebut dengan menampilkan halaman galat "Not Found" (HTTP 404).



Respons penolakan ini memverifikasi bahwa peladen aplikasi telah dikonfigurasi dengan aman. Berkas-berkas kredensial sistem dan arsitektur kode aplikasi tidak berada di dalam direktori publik (*document root*) dan tidak dapat diakses secara paksa melalui peramban, sehingga kerahasiaan data proyek tetap terjamin.

BAB V

KENDALA DAN SOLUSI

Dalam proses perancangan, konfigurasi, dan implementasi infrastruktur layanan web berbasis kontainer, terdapat beberapa kendala teknis yang dihadapi selama fase pengujian operasional. Berikut adalah rincian kendala beserta solusi yang telah diterapkan untuk memastikan sistem berjalan dengan aman dan stabil.

1. Kesalahan Logika pada Otomatisasi Aturan Firewall (UFW)

Pada tahap pra-konfigurasi server yang diotomatisasi menggunakan skrip *bash* (*setup.sh*), ditemukan kesalahan logika sintaks pada pengaturan Uncomplicated Firewall (UFW). Skrip awal secara tidak sengaja mengeksekusi perintah pengizinan lalu lintas masuk secara universal, yang berisiko membatalkan kebijakan isolasi jaringan dan mengekspos peladen ke akses publik yang tidak sah.

Berkas skrip instalasi direvisi agar dapat menerapkan kebijakan bawaan untuk menolak seluruh lalu lintas jaringan masuk (*default deny incoming*). Setelah kebijakan dasar tersebut ditegakkan kembali, pembukaan jalur lalu lintas (*allow rules*) dikonfigurasi ulang secara presisi agar hanya mengekspos *port* yang krusial untuk operasional layanan, yaitu *port* 2222, 80, dan 443.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing

sudo ufw allow 2222/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

sudo ufw reload
```

2. Redundansi Jalur Akses SSH pada Aturan Firewall

Jalur komunikasi akses *remote* SSH telah berhasil dipindahkan dari *port* bawaan ke *port* 2222. Namun, saat dilakukan audit keamanan menggunakan *ufw status verbose*, ditemukan bahwa aturan perizinan untuk *port* 22 bawaan masih berstatus *ALLOW IN*. Hal ini menyisakan celah jaringan yang berpotensi menjadi target pemindaian (*port scanning*).

Solusi untuk mengatasi masalah tersebut adalah dengan mengeliminasi aturan *firewall* yang redundan. Perintah `sudo ufw delete allow 22/tcp` dijalankan untuk menutup *port* 22 secara permanen dari jaringan luar. Verifikasi lanjutan mengonfirmasi bahwa saat ini peladen hanya menerima otentikasi masuk otorisasi melalui *port* 2222.

```
sudo ufw delete allow 22/tcp
```

3. Kesalahan Pengalihan (*Redirect Error*) Akses WordPress ke Host Internal

Saat mengakses peladen melalui peramban menggunakan alamat IP publik, situs tidak dimuat melainkan secara otomatis mengalihkan (*redirect*) URL menjadi `http://app/wp-admin` yang berujung pada error *"site can't be reached"*. Aplikasi WordPress gagal mengenali IP publik dan secara keliru menggunakan nama *host* internalnya (*app*) sebagai *Base URL*.

Untuk mengatasi anomali pengalihan lalu lintas ini, perbaikan dilakukan guna menyinkronkan identitas URL antara *reverse proxy* (Nginx) dan peladen aplikasi (WordPress). Pertama-tama perlu dilakukan modifikasi pada file `default.conf` untuk memastikan berkasnya telah memuat parameter `proxy_set_header Host $host`;

```
proxy_set_header Host $host;  
proxy_set_header X-Real-IP $remote_addr;  
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
proxy_set_header X-Forwarded-Proto $scheme;
```

`/docker/nginx/default.config`

Kemudian pada file konfigurasi WordPress, dilakukan penetapan base URL statis agar aplikasi menggunakan IP Publik dan mengenali enkripsi HTTPS dari Nginx.

```
define('WP_HOME', 'https://172.30.67.104');  
define('WP_SITEURL', 'https://172.30.67.104');  
  
$_SERVER['HTTPS'] = 'on';
```

`wp-config.php`

Setelah itu, server dimuat ulang dengan perintah `sudo docker-compose restart` agar konfigurasi baru dapat diterapkan.

BAB VI

PENUTUP

6.1. Kesimpulan

Berdasarkan hasil perancangan, konfigurasi, dan implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem layanan web berbasis Linux berhasil dibangun dengan menerapkan berbagai mekanisme keamanan server dan jaringan sesuai dengan kebutuhan tugas besar. Implementasi dilakukan menggunakan teknologi containerization melalui Docker yang memisahkan layanan Nginx sebagai reverse proxy, WordPress sebagai application server, dan MariaDB sebagai database server ke dalam lingkungan yang terisolasi.

Penerapan keamanan dilakukan pada berbagai lapisan sistem, mulai dari konfigurasi SSH Hardening dengan penggunaan port kustom dan pembatasan akses, penerapan firewall menggunakan UFW, perlindungan terhadap serangan brute-force menggunakan Fail2Ban, hingga penggunaan protokol HTTPS melalui sertifikat SSL/TLS untuk mengamankan komunikasi data. Selain itu, penggunaan Docker Network dan penyimpanan kredensial melalui file `.env` turut meningkatkan keamanan sistem secara keseluruhan.

Berdasarkan proses implementasi dan verifikasi yang dilakukan, seluruh layanan dapat berjalan dengan baik serta memenuhi kebutuhan fungsional dan keamanan yang telah ditetapkan. Arsitektur yang diterapkan mampu memberikan isolasi layanan, membatasi akses yang tidak diperlukan, serta meningkatkan perlindungan terhadap berbagai potensi ancaman keamanan pada server dan jaringan. Dengan demikian, tujuan dari pelaksanaan tugas besar ini telah berhasil dicapai, yaitu membangun layanan web yang aman, terstruktur, dan menerapkan praktik keamanan yang baik pada lingkungan Linux berbasis kontainer.

LAMPIRAN

Pembagian Tugas Anggota

No.	NIM	Nama Anggota	Tugas
1.	241402036	Ahmad Arif Fatahillah	Menjalankan konfigurasi SSH
2.	241402043	Rahma Sarita Nasution	Membuat file README
3.	241402066	Agnes Olivia Ketaren	Membuat file README
4.	241402073	Richard Lim	Mengonfigurasi dan menyambungkan ke WordPress
5.	241402076	Davina Caecilia Marpaung	Membuat laporan
6.	241402108	Bryan Sullivan Nauli	Uji keamanan
7.	241402117	Jelita Amy Syafira Rangkuti	Membuat desain arsitektur jaringan